task1.py task1_output.txt task2.py task2_output.txt task3.py task3_outp

```python
import os
import time
import hashlib
from concurrent.futures import ThreadPoolExecutor
TASK_COUNT = 10_000
HASH_ROUNDS = 500
PAYLOAD = b"A" * 1024
def cpu_task(task_id):
    data = PAYLOAD
    digest = b""
    for _ in range(HASH_ROUNDS):
        digest = hashlib.sha256(data).digest()
    return digest[0] + task_id % 256
def run_benchmark(pool_size):
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=pool_size) as executor:
        results = list(executor.map(cpu_task, range(TASK_COUNT)))
    elapsed_ms = (time.perf_counter() - start) * 1000
    if len(results) != TASK_COUNT:
        raise RuntimeError("Task count verification failed")
    return elapsed_ms, sum(results)
def main():
    cores = os.cpu_count() or 1
    pool_sizes = [
        1,
        max(1, cores // 2),
        cores,
        cores * 2,
        cores * 4,
        cores * 16,
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py

```
22  def main():
29          cores * 4,
30          cores * 16,
31          256
32      ]
33      pool_sizes = sorted(set(pool_sizes))
34      print("TASK 5 — Thread Oversubscription Threshold")
35      print(f"Logical CPU cores: {cores}")
36      print(f"CPU-bound tasks: {TASK_COUNT:,}")
37      print(f"SHA-256 rounds per task: {HASH_ROUNDS}")
38      print(f"Unique pool sizes: {pool_sizes}")
39      results = []
40      for pool_size in pool_sizes:
41          elapsed_ms, checksum = run_benchmark(pool_size)
42          results.append((pool_size, elapsed_ms, checksum))
43          print(f"Completed pool size {pool_size}: {elapsed_ms:.2f} ms")
44      baseline_time = results[0][1]
45      first_worsening = None
46      for i in range(1, len(results)):
47          if results[i][1] > results[i - 1][1]:
48              first_worsening = results[i][0]
49              break
50      lines = [
51          "TASK 5 — Thread Oversubscription Threshold",
52          f"Logical CPU cores: {cores}",
53          f"CPU-bound tasks: {TASK_COUNT}",
54          f"SHA-256 rounds per task: {HASH_ROUNDS}",
55          "",
56          f"{'Pool Size':>12} | {'Execution Time (ms)':>20} | "
57          f"{'Speedup':>10} | Status",
```

```
22      def main():
56          f"{'Pool Size':>12} | {'Execution Time (ms)':>20} | "
57          f"{'Speedup':>10} | Status",
58          "-" * 75
59      ]
60
61      for i, (pool_size, elapsed_ms, checksum) in enumerate(results):
62          speedup = baseline_time / elapsed_ms if elapsed_ms > 0 else 0.0
63          if pool_size == first_worsening:
64              status = "FIRST WORSENING"
65          elif i > 0 and elapsed_ms > results[i - 1][1]:
66              status = "SLOWER"
67          else:
68              status = "OK"
69          lines.append(
70              f"{pool_size:12d} | {elapsed_ms:20.2f} | "
71              f"{speedup:9.2f}x | {status}"
72          )
73      lines.append("")
74      if first_worsening is not None:
75          lines.append(
76              f"First observed worsening: pool size {first_worsening}"
77          )
78          lines.append(
79              "This is the first tested pool size slower than the previous "
80              "tested size."
81          )
82      else:
83          lines.append(
84              "No worsening observed between consecutive tested pool sizes."
```

```
    else:
        lines.append(
            "No worsening observed between consecutive tested pool sizes."
        )

    lines.append(
        "Note: timing alone cannot prove that OS context switching "
        "is the sole cause."
    )

    output = "\n".join(lines)

    with open("task5_output.txt", "w", encoding="utf-8") as file:
        file.write(output + "\n")

    print("\n" + output)
    print("\nResults saved to task5_output.txt")


if __name__ == "__main__":
    main()
```